

Scratch for Parents: Learn It, Then Teach It

Your working notebook for the whole workshop series — all five modules.
Read the key ideas, follow the build steps, try the Teach-It practice, write in
your learning journal, and check yourself with the quizzes.

5 modules • 17 lessons

Build steps, journal & self-checks

Name: _____

CONTENTS

MOD 1 Getting Started with Scratch

- 1.1 What Is Scratch & Why Kids Love It
- 1.2 A Guided Tour of the Scratch Editor
- 1.3 Your First Project — Make the Cat Move & Talk

MOD 2 Core Coding Concepts

- 2.1 Sequences & Timing
- 2.2 Loops — Making Things Repeat
- 2.3 Conditionals & Sensing — Making Choices

MOD 3 Building Real Projects

- 3.1 An Interactive Animated Story
- 3.2 A Simple Game with Variables & Score
- 3.3 Debugging & Remixing — Fixing and Improving

MOD 4 Leveling Up — Bigger Projects

- 4.1 Clones — One Sprite, Many Copies
- 4.2 My Blocks — Make Your Own Blocks
- 4.3 Capstone Part 1 — Plan It & Build the Core
- 4.4 Capstone Part 2 — Lives, Game Over & Launch Day

MOD 5 Teaching Scratch to Your Child

- 5.1 The Art of Not Taking Over
- 5.2 Sharing Safely & the Scratch Community
- 5.3 ScratchJr — Coding for Ages 5–7

5.4 What Comes After Scratch

Getting Started with Scratch

From "what is this?" to "I made that!" You'll learn what Scratch is, find your way around the editor, and build your first program: a cat that walks and says hello. You'll also practice how to hand each step over to your child.

Lessons 1.1-1.3

3 lessons

Session S1

Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **Key Ideas**, a **Worked Example**, a **Your Turn** build, **Teach-It Practice** for the role-play with your partner, a short **Self-Check**, and **Learning Journal** prompts. Write honestly, not neatly. Nobody grades this. **You can't break anything in Scratch**, so click freely.

ACCOUNT OR NO ACCOUNT?

You **don't need an account** today. Go to scratch.mit.edu → **Create** and start. To keep your work without an account, use **File → Save to your computer** (it downloads a `.sb3` file). With a free account, **File → Save now** saves online. New accounts must **confirm their email** (check spam) before they can share. If the website is blocked where you are, the free **Scratch app** from scratch.mit.edu/download works offline and looks the same.

MODULE PROJECT TRACKER

- ◆ ☐ **1.1** — journal "why page" written; I can say what Scratch is in one sentence
- ◆ ☐ **1.2** — I can name the four zones; I saved a project and reopened it
- ◆ ☐ **1.3** — my cat walks and talks when I click the green flag
- ◆ ☐ **At home** — my child pressed GO on their own cat program

◆ BY THE END YOU CAN

- ◆ Explain in one sentence what Scratch is
- ◆ Say how block coding differs from typed code
- ◆ Introduce Scratch to your child with confidence

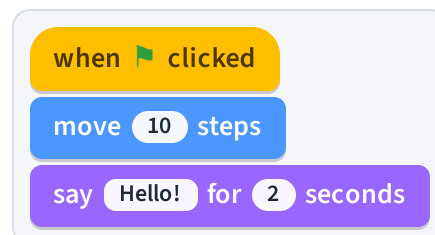
◆ KEY IDEAS

- ◆ **Scratch** is a free website where kids make games, animations and stories by **snapping colorful blocks together** instead of typing code. It's made by MIT and run by the non-profit Scratch Foundation.
- ◆ Three words: **sprite** (a character or object, like the cat), **stage** (the screen where the action happens), **blocks** (the instructions you snap together).
- ◆ Blocks only snap together if they make sense, so there are **no typos to break things**.
- ◆ **Color = category**: blue Motion, purple Looks, pink Sound, gold Events, orange Control, light-blue Sensing, green Operators, dark-orange Variables.
- ◆ You are a **coding buddy**, not an expert. Add the word "**yet**": "I don't get it yet."

◆ WORKED EXAMPLE – SAME PROGRAM, TWO WAYS

```
# typed code (Python-style)
when program starts:
    move(10)
    say("Hello!", 2)
```

Miss a colon or misspell `say` and it breaks.



Snaps together like LEGO. Nothing to misspell.

◆ YOUR TURN – CLICK AROUND

- 1 Open scratch.mit.edu and click **Create**. The cat is waiting.
- 2 Drag *any* block into the middle and click it. What happened?
- 3 Drag one block onto another until it snaps. Try to snap a gold "when..." block *under* something. Notice that it won't go.

One thing that surprised me:

◆ TEACH-IT PRACTICE – "THE PITCH"

Your partner plays the child on the kid card. Invite them to try Scratch in under a minute, with no screen. Lead with *their* interests. **"Make" beats "learn."**

SAY: *"Want to make your own game where you decide the rules? There's a free thing called Scratch where you snap colorful blocks together and a little cat does whatever you tell it. Want to try making it talk?"*

SAY: *"I'm learning it too, so I'll probably get stuck. Maybe you can help me."*

My pitch for **my** child (use what they love right now):

Role-play reflection: which line made my "child" curious? Which one sounded like school?

◆ SELF-CHECK

Q1

What is the single biggest advantage of block-based coding for children?

- A) Programs run faster than typed code
- B) You can't make syntax/typing errors that break the program
- C) It costs less money

Q2

In Scratch, what does a block's color tell you?

- A) How hard the block is to use
- B) Which category it belongs to (Motion, Looks, Events...)
- C) The order it must run in

Q3

What's the best first-session goal with your child?

A) Finish a complete game

B) Memorize the coding vocabulary

C) Create one moment of "whoa, I made that!"

Q4

In Scratch, the orange cat is an example of a...

A) Stage

B) Sprite

C) Block

◆ LEARNING JOURNAL – YOUR "WHY PAGE"

Why do you want to learn this? What do you hope to create with your child?

What worries you about it? Name the worry. You'll cross it off later.

In one sentence of your own, what is Scratch? What do **sprite**, **stage** and **blocks** mean?

Confidence helping with Scratch right now (1–10): _____ (you'll compare this at the end of the course)

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Name the four areas of the editor and what each does
- ◆ Find any block with the colored category menu
- ◆ Run and stop a project, and save your work

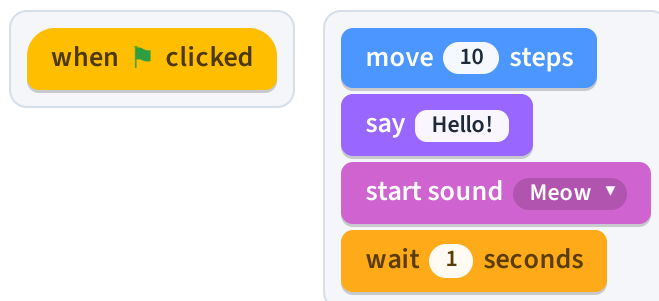
◆ KEY IDEAS

#	ZONE	WHERE	WHAT IT'S FOR
1	Blocks Palette	Left	The menu of instructions, sorted by color/category
2	Code Area	Middle	Where you drag and snap blocks together
3	Stage	Upper right	The "screen" where sprites act out your code
4	Sprite List	Lower right	All your characters; click one to code it

- ◆ **Green flag** runs every **when**  **clicked** script. **Red stop** halts everything.
- ◆ **Code belongs to the selected sprite**. If "nothing works", first check which sprite is highlighted.
- ◆ Tabs at the top-left: **Code** (blocks), **Costumes** (a sprite's outfits, which is how animation works), **Sounds**.
- ◆ **Saving**: signed in → **File** → **Save now**. Not signed in → **File** → **Save to your computer**, and reopen it later with **File** → **Load from your computer**.
- ◆ **Kid accounts**: use **Join Scratch**, a username that hides their real name, and **your** email as parent/guardian. Then click the confirmation link in that email.

◆ WORKED EXAMPLE – THE BLOCK YOU'LL USE MOST

Gold **Events** holds the starter block. Almost every program begins with it:



Right: one block each from Motion, Looks, Sound and Control. You can recognize them by color before you read the words.

◆ YOUR TURN – POINT AND NAME, THEN SAVE & RELOAD

- 1 Play "Point and Name" with the room, then with your partner: *"Find a block that plays a sound."* *"Find the block that starts a program."* *"How many colored dots are there?"*
- 2 Press the green flag and the red stop a few times.
- 3 Click the cat's **Costumes** tab. How many costumes does it have? ____
- 4 **File → Save to your computer**. Find the **.sb3** file (usually in Downloads). Then **File → Load from your computer** to reopen it.

Sketch the editor and label the four zones:

◆ TEACH-IT PRACTICE – "THE 60-SECOND TOUR"

Your partner (the "child") holds the mouse. You give the tour with **your hands in your lap**. Point with a finger, ask, and let them click.

SAY: *"See these four parts? Over here are the blocks. That's your toy box. In the middle is where you build. Up here is where the cat does what you say. And this green flag makes it GO. Want to press it?"*

SAY: *"How many colors of block can you find? Let's count the dots."*

SAY: *"Can you teach me the four parts?"*

Role-play reflection: how many times did my hand go for the mouse? What helped me stop?

◆ SELF-CHECK

Q1

Your child dragged in some blocks but nothing happens when they run it. What's the most likely cause?

- A) Scratch needs a paid upgrade
- B) The code is on a different sprite than the one selected
- C) The internet is too slow

Q2

Which button runs the project?

- A) The green flag
- B) The red stop
- C) The Costumes tab

Q3

In the editor, what does a block's category color help you do?

- A) Make the sprite move faster
- B) Quickly find the kind of block you need in the palette
- C) Save the project

Q4

You're not signed in to Scratch. How do you keep today's project?

- A) You can't. You need an account
- B) It saves automatically
- C) File → Save to your computer

◆ LEARNING JOURNAL

What clicked: which part suddenly made sense? The color categories? That code belongs to the selected sprite?

Questions & confusion: which button, tab or menu are you still unsure about?

Account decision: kid account now, or save files to the computer for a while? Why?

◆ **NOTES**

◆ BY THE END YOU CAN

- ◆ Drag, drop and snap blocks into a script
- ◆ Start a program with **when**  **clicked**
- ◆ Test, tweak and fix a simple project
- ◆ Coach your child with questions, not instructions

◆ KEY IDEAS

- ◆ A program starts with an **Event** ("hat") block, like **when**  **clicked**.
- ◆ Blocks **snap** into a stack. Watch for the grey shadow before you let go.
- ◆ Scratch runs a stack **top to bottom, in order**. That's a **sequence**. Change the order and you change the behavior.
- ◆ Click a **white oval** to change words or numbers.
- ◆ Bugs are normal: "It doesn't work... yet."

◆ WORKED EXAMPLE – BUILD IT ONE BLOCK AT A TIME



◆ YOUR TURN – MAKE THE CAT MOVE & TALK

- 1 New project (**File → New**). Check that the cat (Sprite1) is **selected** in the sprite list.
- 2 **Events** (gold): drag **when**  **clicked** into the code area.
- 3 **Motion** (blue): snap **move 10 steps** underneath. Click the green flag.
- 4 **Looks** (purple): snap **say Hello! for 2 seconds** under the move block.
- 5 Change the message to your own words and **10** to **50**. Run it!
- 6 Cat stuck at the edge? Add **go to x: 0 y: 0** right under the hat.
- 7 **Make it yours** (pick any): a joke message · 200 steps · a second **say** · **start sound Meow** · **change size by 25** · **turn  15 degrees** · **next costume** between move and say.

8 **Save it** (File → Save now, or Save to your computer).

What happened when I swapped **move** and **say**?

◆ IF IT DOESN'T WORK

- ◆ **Nothing happens on the green flag** → the blocks aren't snapped (drag the hat and see whether everything comes with it), or the wrong sprite is selected.
- ◆ **Cat moves but doesn't talk** → the **say** block isn't attached. Drag it until it snaps under **move**.
- ◆ **Cat walks off-screen** → each run adds 50 more steps. Drag the cat back, or add **go to x: 0 y: 0**.
- ◆ **Cat is upside-down** after turning → set its rotation to "left-right" in the sprite info.

◆ TEACH-IT PRACTICE – "TREASURE HUNT, NOT DICTATION"

Coach your partner through the build using **questions only**. The words, the number and the green flag belong to the child.

SAY: *"We want the cat to walk. Which color do you think has the moving blocks?"*

SAY: *"The message is yours. What should the cat say?"*

SAY: *"Hmm, it didn't work yet. Interesting! Why do you think it did that?"*

Role-play reflection: when my "child" got stuck, what did I say? What would I say differently next time?

◆ SELF-CHECK

Q1

Which block should be at the top of the script so it runs when you press the green flag?

A) move 10 steps

B) when  clicked

C) say Hello!

Q2

Scratch runs the blocks in a stack...

A) from top to bottom, in order

B) in a random order

C) from bottom to top

Q3

Your child's cat won't talk, though it moves. Best first check?

A) Restart the computer

B) Is the "say" block snapped onto the stack?

C) Buy the full version

Q4

You drag say ... for 2 seconds *above* move 50 steps (still under the hat). What happens now?

A) Nothing. Scratch picks its own order

B) The cat walks and talks at the same time

C) The cat talks first, then walks

◆ LEARNING JOURNAL

Key concepts: in your own words, what is a sequence? Why does the order of blocks matter?

Ideas to try: one thing you'd like the cat to do that you don't know how to do yet.

Progress & feelings: how did it feel to write your first program? Confidence now (1–10): _____

◆ NOTES

TRY AT HOME THIS WEEK

- ◆ Have the "want to make your own game?" chat with your child.
- ◆ Let them build the cat program themselves. **Hands in your lap**; they press GO every time.
- ◆ Give them a "coder's notebook" to draw what their sprite did.
- ◆ Optional: create a kid account together and confirm the email.
- ◆ Have them show the project to someone: a sibling, a grandparent, or someone on a video call.

YOU FINISHED MODULE 1 🎉

You went from "what is this?" to building a real program, and you've rehearsed how to hand it over to your child. You are officially their coding buddy. In Module 2 the cat stops doing one thing and starts *behaving*, with timing, loops and choices.

Core Coding Concepts

Three ideas sit under every program ever written: steps in order, steps that repeat, and steps that choose. In this session you'll learn each one with a game away from the screen, then build it on your cat. You'll also practise how to teach it to your child.

Lessons 2.1-2.3

3 lessons

Session S2

Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **Key Ideas**, a **Worked Example** (the blocks drawn in Scratch colors), **Your Turn** build steps, a **Teach-It Practice** for the pair role-play, a **Self-Check**, and a **Learning Journal**. If your script does something surprising, you haven't failed. The computer is showing you exactly what your steps say.

MODULE PROJECT TRACKER

- ◆ ☐ **Warm-up** — "programmed" the facilitator out the door
- ◆ ☐ **2.1** — a timed greeting routine (sequence + wait)
- ◆ ☐ **2.2** — a cat that really walks (repeat loop + costumes)
- ◆ ☐ **2.3** — a cat I drive with the arrow keys (forever + if)
- ◆ ☐ **At home** — played the robot game and an if/then round with my child

◆ BY THE END YOU CAN

- ◆ Explain what a sequence is and why order matters
- ◆ Use the **wait** block to control timing and pacing
- ◆ Build a short multi-step routine that runs like a little performance

◆ WARM-UP – PROGRAM THE FACILITATOR

Write the class's final instructions that got the "robot" out the door. Circle any step that got repeated over and over.

◆ KEY IDEAS

- ◆ A **sequence** is a list of steps run one after another, **top to bottom**. Think of a recipe.
- ◆ Computers are **literal**. Same blocks in a different order = different behavior. A surprise usually means the recipe isn't what you intended yet.
- ◆ **wait (1) seconds** (orange **Control**) adds pacing. Decimals like 0.5 work.
- ◆ **say ... for 2 seconds** already pauses before the next block. Plain **say ...** moves straight on.

◆ WORKED EXAMPLE – ORDER CHANGES EVERYTHING

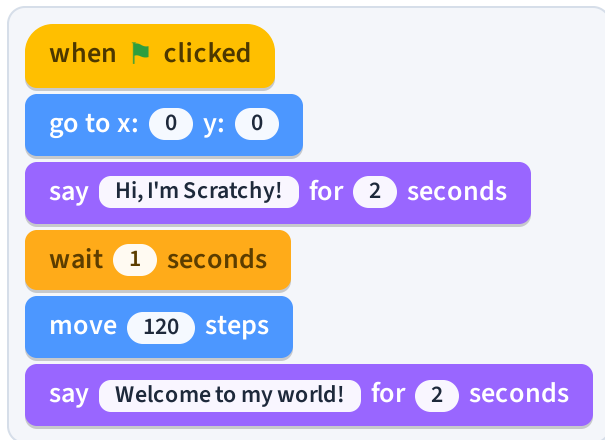


Left: walks, then speaks. Right: speaks, then walks. The blocks are the same and only the order is different.

◆ YOUR TURN – BUILD A GREETING ROUTINE

- 1 Start with **when green flag clicked** (Events).
- 2 Add **go to x: 0 y: 0** (Motion) so the cat always starts centered.
- 3 Add **say "Hi, I'm Scratchy!" for 2 seconds** (Looks), then **wait 1 seconds** (Control).
- 4 Add **move 120 steps**, then **say "Welcome to my world!" for 2 seconds**.

5 **Challenge:** between the wait and the move, add **start sound Meow** and **next costume**.



◆ TEACH-IT PRACTICE – THE ROBOT GAME

You're the robot. Your "child" gives you steps to make a sandwich, and you follow them exactly as said.

SAY: *I'm a robot. I only do exactly what you say, one step at a time. What's my first step?*

SAY: *The computer is a robot just like I was. If it looks wrong, let's read our steps in order and find the mix-up.*

Reflection: when your "child" got frustrated, what did you say or do that helped?

◆ SELF - CHECK

Q1

What is a "sequence" in coding?

A) Steps carried out in order, one after another

B) Steps run in a random order

C) A step that repeats forever

Q2

Which category holds the wait block?

A) Motion (blue)

B) Control (orange)

C) Looks (purple)

Q3

Your child's cat "freezes" for two seconds mid-routine. Most likely reason?

A) The computer is broken

B) A "say for 2 seconds" or "wait" block is holding, as designed

C) The cat is moving too fast

Q4

What's the difference between say Hello! for 2 seconds and plain say Hello!?

A) None — they're the same block

B) The timed one pauses the script before the next block; plain say moves straight on

C) Plain say only works on the stage

◆ LEARNING JOURNAL

- ◆ **Key concepts:** Explain a sequence using a recipe or routine from your own day.
- ◆ **What clicked:** When you swapped the move and say blocks, what surprised you?
- ◆ **Ideas to try:** A short scene (a joke, a greeting) you could turn into a timed sequence with your child.

◆ NOTES

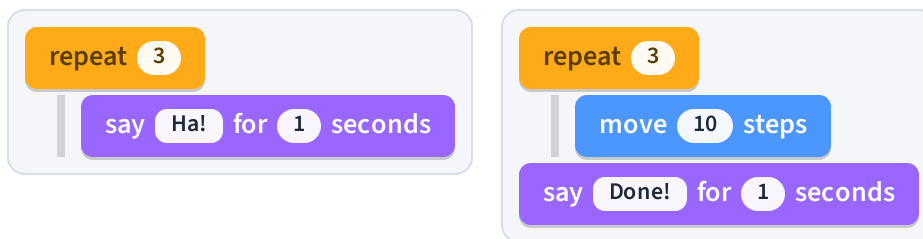
◆ BY THE END YOU CAN

- ◆ Use the **repeat** and **forever** loops
- ◆ Tell what goes **inside** a loop vs **outside** it
- ◆ Build a smooth walking animation, and stop a runaway forever loop

◆ KEY IDEAS

- ◆ A loop says "do this again and again." One loop replaces a pile of copied blocks.
- ◆ **repeat (10)** = a set number of times, then move on. **forever** = until you press the red stop sign.
- ◆ Loops are **C-shaped**. **Only the blocks inside the mouth repeat.**
- ◆ Anything placed *after* a forever loop never runs. That's the loop doing its job.

◆ WORKED EXAMPLE — INSIDE VS OUTSIDE



Left: "Ha!" three times. Right: moves three times, then says "Done!" **once**.

◆ YOUR TURN — A REAL WALKING CAT

- 1 **when green flag clicked, then go to x: -200 y: 0** (start at the left).
- 2 Add a **repeat 20** loop (Control).
- 3 Inside the loop: **move 20 steps, next costume, wait 0.1 seconds**.
- 4 Press the green flag. Try a wait of 0.3, then 0.05. What changes?
- 5 **Challenge:** wrap the walk in **forever**, add **if on edge, bounce** and **set rotation style [left-right]**.



◆ TEACH-IT PRACTICE – SPOT THE LOOP

Your "child" has copied the same block fifteen times. Don't fix it. Ask.

SAY: *Brushing teeth is a loop: scrub, scrub, scrub, repeat until clean! Can you think of another loop?*

SAY: *That works! Is there a block that could do all of those for us at once, so we don't have to make so many?*

SAY: *Is your block inside the mouth of the C, or outside?*

Reflection: did your "child" find the loop, or did you end up showing them? What would you try differently?

◆ SELF-CHECK

Q1

You want the cat to blink exactly 5 times. Best loop?

A) repeat 5

B) forever

C) wait 5 seconds

Q2

Only which blocks repeat in a loop?

A) All blocks in the whole script

B) Only the blocks inside the C-shape

C) Only the blue ones

Q3

A block placed after a "forever" loop will...

A) never run

B) run first

C) alternate with the loop

Q4

Your walking cat zooms across the stage in a blink. Best first fix?

A) Add (or lengthen) a wait inside the loop

B) Put a wait after the loop

C) Change repeat to forever

◆ LEARNING JOURNAL

- ◆ **Key concepts:** In your own words, repeat vs forever. Why does it matter which blocks sit inside the C? (A sketch is a fine answer.)
- ◆ **Questions & confusion:** What still feels tricky? Loops inside loops? How wait changes speed?
- ◆ **Progress & feelings:** What everyday "loops" could you spot with your child this week?

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Use **if/then** and **if/then/else** to make decisions
- ◆ Read Sensing blocks (touching, key pressed, mouse down)
- ◆ Combine **forever** + **if** to check things constantly, and drive the cat with the arrow keys

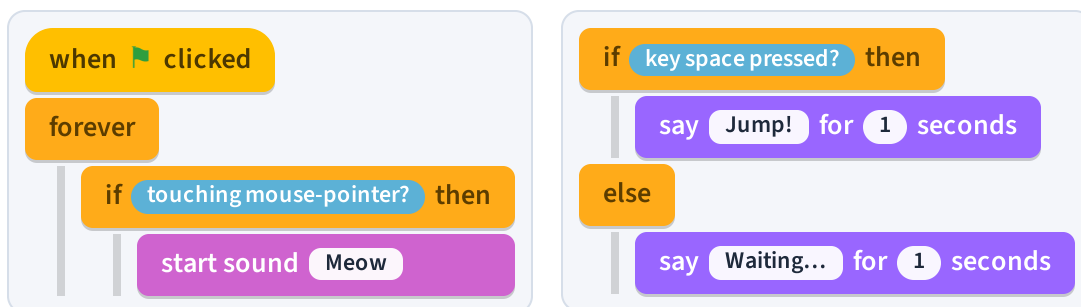
◆ WARM-UP — "IF IT'S RAINING, THEN..."

Write two rules from your own household: one **if/then**, and one **if/then/otherwise**.

◆ KEY IDEAS

- ◆ **if <> then** (Control) runs the blocks inside **only when the condition is true**. Otherwise Scratch skips them.
- ◆ The pointy **hexagon slot** takes a true/false question. Most come from light-blue **Sensing**.
- ◆ **forever** + **if** is the "game loop": it keeps asking, so the sprite reacts the instant something becomes true. Without forever, the check happens once.
- ◆ **if/then/else** has two mouths. Read "else" as "otherwise."

◆ WORKED EXAMPLE — THE GAME LOOP AND TWO PATHS

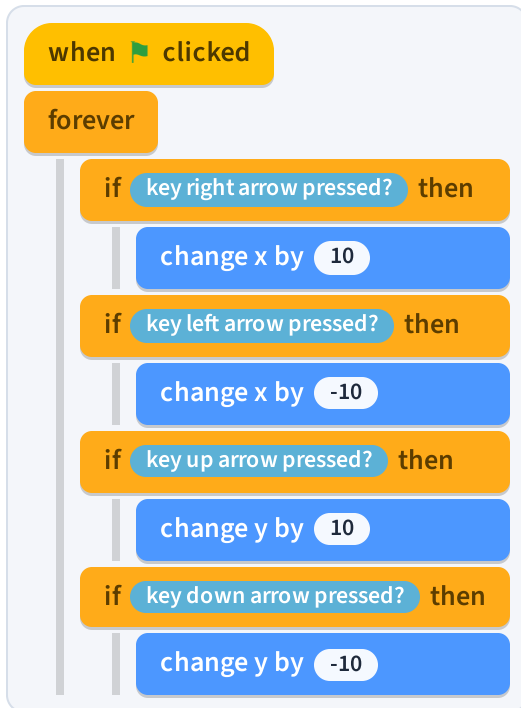


Read them aloud: "Forever: if touching the pointer, meow." "If space is pressed, say Jump. Otherwise, say Waiting."

◆ YOUR TURN — DRIVE THE CAT WITH ARROW KEYS

- 1 **when green flag clicked**, then a **forever** loop.
- 2 Inside the forever, add **if <key right arrow pressed?> then** with **change x by 10** inside. Test it.

- 3 Right-click the if → **Duplicate** three times. Change each to left (x -10), up (y 10), down (y -10).
- 4 Press the green flag, **click the stage once**, and drive.
- 5 **Challenge:** add if <touching edge?> then say "Ouch!".



◆ TEACH-IT PRACTICE – COACH WITH QUESTIONS

Your "child" has the mouse. Keep your hands in your lap.

SAY: *If it's raining, THEN we bring an umbrella. Otherwise we don't. Give me an "if/then" rule from your day!*

SAY: *How does the cat know you pressed a key? Which color has "senses"?*

SAY: *We want it to check all the time. What loop does that?*

Reflection: how hard was it to keep your hands off the mouse? Which question worked best?

◆ SELF-CHECK

Q1

What shape of block fits in an "if ____ then" slot?

A) A hexagon (true/false) block

B) An oval reporter block

C) A hat block

Q2

Why do most games wrap their "if" checks in a forever loop?

A) To make the game faster

B) So the condition is checked constantly, not just once

C) To change the block colors

Q3

"if/then/else" is best when...

A) you only act when something is true

B) you want one thing if true and a different thing otherwise

C) you want to repeat something

Q4

Your arrow-key script matches the example exactly, but the cat won't move. What do you try first?

A) Click the stage once so it has keyboard focus

B) Restart the computer

C) Delete the forever loop

◆ LEARNING JOURNAL

- ◆ **Key concepts:** What is a condition? Why does an if almost always need a forever around it?
- ◆ **What clicked:** Reading an if aloud as a sentence? The moment the cat first moved?

- ◆ **Progress & feelings:** You've now met sequences, loops, and conditionals. How confident do you feel compared with Session 1?

◆ NOTES

TRY AT HOME THIS WEEK

Play the **robot game** (you're the robot, and your child programs you to make a snack). Play a round of "**if/then**" **rules** at dinner or in the car. Show your child the **arrow-key cat** and let them change the numbers. Save it, because you'll reuse this movement code in Module 3.

YOU FINISHED MODULE 2 🎉

Sequences, loops, and conditionals are the three ideas behind all programming, in any language. Everything ahead builds on them. Next session: a two-character animated story and your first real game with a score.

Building Real Projects

Today you build the things kids beg for: an animated story where characters take turns talking, and a real game with a score. Then you learn the skill that makes someone a real coder: calmly fixing what goes wrong.

Lessons 3.1–3.3

3 lessons

Session S3

Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **Key Ideas**, a **Worked Example** (the actual blocks), a **Your Turn** build, **Teach-It Practice** for the role-play, a **Self-Check**, and a **Learning Journal**. If a build feels hard today, that's normal. This is the biggest step up in the course. Get one piece working, then the next.

MODULE PROJECT TRACKER

- ◆ ☐ 3.1 A two-character conversation that plays itself (broadcasts)
- ◆ ☐ 3.1 *Bonus*: the Stage changes the backdrop mid-story
- ◆ ☐ 3.2 Catch the Apple, a playable game with a live score (saved!)
- ◆ ☐ 3.3 Bug Hunt: all four planted bugs found and fixed
- ◆ ☐ 3.3 Explored one shared project with **See Inside**

◆ BY THE END YOU CAN

- ◆ Add sprites and a backdrop from the libraries
- ◆ Place sprites with `go to` and move them smoothly with `glide`
- ◆ Make characters take turns with `broadcast` and `when I receive`

◆ KEY IDEAS

- ◆ The stage is a grid: x is left-right (-240 to 240), y is down-up (-180 to 180), and the centre is 0, 0. Drag a sprite and read its x/y under the stage.
- ◆ `go to x: y:` **teleports** (good for a starting spot). `glide _ secs to x: y:` **slides** (good for entrances).
- ◆ Sprites can't see each other's scripts. A **broadcast** is an invisible signal that any sprite can send and any sprite can listen for, like a stage director calling a cue.
- ◆ Only the **first** line starts with `when clicked`. Every later line starts with `when I receive`.

◆ WORKED EXAMPLE

The image shows two Scratch scripts. The left script, for the Cat, starts with a gold 'when clicked' block, followed by a blue 'go to x: -200 y: -100' block, a blue 'glide 2 secs to x: 0 y: -100' block, a purple 'say Hi Bear! for 2 seconds' block, and a gold 'broadcast yourturn' block. The right script, for the Bear, starts with a gold 'when I receive yourturn' block, followed by a purple 'say Hi Cat! Nice day! for 2 seconds' block.

Left: the Cat's script (sends the cue). Right: the Bear's script (listens for it). Both gold blocks are in **Events**. Choose **New message** the first time and give the cue a clear name.

◆ YOUR TURN – A CONVERSATION THAT PLAYS ITSELF

- 1 New project. Choose a **Backdrop** (bottom-right) and pick a setting. Choose a **Sprite** and add a second character.
- 2 Cat (`when clicked`): say a line, then `broadcast [line 2]`.
- 3 Other character (`when I receive [line 2]`): reply, then `broadcast [line 3]`.
- 4 Cat (`when I receive [line 3]`): reply, then `broadcast [line 4]`.

5 Both (`when I receive [line 4]`): say goodbye and `glide` off together.

6 *Bonus:* click the **Stage** and give it `when I receive [night]` → `switch backdrop to` a night scene. Have a character broadcast "night."

Plan your four lines here first (who says what?):

◆ TEACH-IT PRACTICE – STORY BEFORE CODE

The 3-step story warm-up: **Who?** (two characters, with names) → **Where?** (the backdrop) → **What happens?** (act it out loud in voices, and that becomes your script).

SAY: *Let's be the characters first! You be the bear, I'll be the cat. What does the cat say? ...Great, now let's make Scratch say exactly that.*

SAY: *Pick any two you like. We can come back to the library later. Who's our hero today?*

Role-play reflection: What did the "child" say that surprised you? Which line got them talking?

◆ SELF-CHECK

Q1

Two sprites need to take turns talking. What coordinates them?

A) broadcast and "when I receive"

B) Carefully matched wait blocks

C) A forever loop on each

Q2

Which block moves a sprite smoothly over time?

A) go to x/y

B) glide _ secs to x/y

C) next costume

Q3

Both characters talk at the exact same time. Likely cause?

A) Both start with "when  clicked" instead of "when I receive"

B) The backdrop is wrong

C) The sprites are too big

Q4

You want the scene to turn to night when a sprite broadcasts "night." Where does the backdrop-changing script go?

A) On the sprite, under "when  clicked"

B) On the Stage: "when I receive night" then "switch backdrop to"

C) You can't change backdrops during a project

◆ LEARNING JOURNAL

Key concepts: Explain a broadcast in your own words, as if to a friend. How is it different from lining up wait blocks?

What clicked / what's fuzzy: The x/y grid? The glide entrance? Which scripts start with the flag and which with "when I receive"?

Ideas to try: Sketch a story to make with your child: who's in it, where, and one scene change.

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Explain what a variable is and make one
- ◆ Keep score with `set` and `change`, and show it on the stage
- ◆ Combine loops, `if`, sensing, and operators into a real game

◆ KEY IDEAS

- ◆ A **variable** is a labeled box (or a whiteboard) that holds a value the program can remember and change: a score, lives, a name.
- ◆ **Make a Variable** is in the orange **Variables** category. Name it `score` and keep **For all sprites**. Tick its checkbox to show it live on the stage.
- ◆ `set [score] to 0` replaces the value. `change [score] by 1` adds to it. The round `(score)` reporter reads it.
- ◆ Reset the score **once**, right after the green flag, **before** the game loop.
- ◆ Why check `y position < -170` instead of `touching edge`? The apple starts at y: 170, where it's already touching the **top** edge, so "touching edge" would send it back to the top every frame and it would never fall.

◆ WORKED EXAMPLE

The image shows two Scratch code snippets. The left snippet is for an 'Apple' sprite, and the right snippet is for a 'Cat' sprite.

Left: Apple's code:

- `when green flag clicked`
- `set score to 0`
- `forever` loop:
 - `go to x: pick random -200 to 200 y: 170`
 - `repeat until touching Cat?`
 - `change y by -8`
 - `if y position < -170 then`
 - `go to x: pick random -200 to 200 y: 170`
 - `change score by 1`

Right: Cat's code:

- `when green flag clicked`
- `forever` loop:
 - `go to x: mouse x y: -140`

Left: the **Apple**. Start at the top at a random x, fall until caught (jump back to the top if it drops past the bottom), then add a point, forever. Right: the **Cat** follows the mouse left-right along the bottom. Green =

Operators, blue-green = Sensing. The bottom check puts the **y position** reporter (bottom of Motion) inside the green < block.

◆ YOUR TURN – CATCH THE APPLE

- 1 Add an **Apple** sprite (keep the Cat). Make the **score** variable and tick its checkbox.
- 2 Build the Apple script. Watch the white landing shadow so each block drops into the right C-shaped mouth.
- 3 Build the Cat script. **mouse x** is a round **Sensing** block that you drop into the x slot.
- 4 Press the green flag and play. Then **save**: File → Save to your computer. You'll need it again.
- 5 *Challenge*: a **time left** variable set to 30 that counts down by 1 each second, then **broadcast [game over]** and **stop all**.

◆ TEACH-IT PRACTICE – DESIGN FIRST, CODE SECOND

The one-page game design: **Goal** (catch apples) · **Controls** (move the mouse) · **Scoring** (+1 per catch) · **Lose condition** (timer runs out). Explain variables with a sticky note that says "SCORE: 0" and cross it out each time.

SAY: *Before we code, tell me your game like you're explaining it to a friend. How do you play? How do you win? Let's write those down. Those are our to-do list.*

SAY: *Let's put that on the "someday" list and make catching one thing perfect first.*

Role-play reflection: How did you steer the "child's" huge idea down to one mechanic without saying "no"?

◆ SELF-CHECK

Q1

What is a variable?

A) A named box that stores a value the program can change

B) A block that repeats actions

C) A character on the stage

Q2

To add one point to the score, you use...

A) set score to 1

B) change score by 1

C) set score to 0

Q3

Where should "set score to 0" go?

A) Right after the green flag, before the game loop

B) Inside the forever loop

C) It doesn't matter

Q4

You made the **score** variable but can't see it on the stage. What do you do?

A) Make a second variable

B) Add a say block to every sprite

C) Tick the checkbox next to the variable in the Variables palette

◆ LEARNING JOURNAL

Key concepts: Describe a variable with your own everyday comparison. What's the difference between *set* and *change*?

Questions & confusion: Does the **repeat until** inside the **forever** make sense yet? Which part would you re-read?

Progress & feelings: This was the biggest build so far. How did it feel to juggle it all?

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Find and fix bugs with the 4-step method
- ◆ Recognize the most common Scratch bugs
- ◆ Use **See Inside** and explain good remixing etiquette
- ◆ Coach a frustrated child without grabbing the mouse

◆ KEY IDEAS

- ◆ A bug just means "not doing what I expected yet." The computer does exactly what the blocks say. Debugging is finding the gap between what you said and what you meant.
- ◆ The 4-step loop: **1 Watch** what actually happens → **2 Predict** which block causes it → **3 Change ONE thing** → **4 Test**. Still wrong? Go round again.
- ◆ Scratch's helpers: click one block to run just that piece, tick a variable's checkbox to watch it, add a temporary `say (score)`, slow things down with `wait`.
- ◆ **See Inside** opens any shared project's code. **Remix** (when signed in) makes your own copy. Scratch credits the original creator automatically. Change something meaningful and keep the credit.

◆ WORKED EXAMPLE – THE USUAL SUSPECTS

SYMPTOM	USUAL CAUSE	FIX
Nothing happens	Code is on the wrong sprite	Select the right sprite in the sprite list
Runs once, never reacts again	An <code>if</code> with no loop around it	Wrap the check in a <code>forever</code>
Blocks after a loop never run	Trapped below a <code>forever</code>	Move them inside, or use <code>repeat until</code>
Sprite zooms off / upside-down	Position not reset / rotation style	Add <code>go to x/y</code> at start; set rotation style to left-right
Score jumps by many at once	Condition stays true for many frames	Move the sprite away, or add a short wait after scoring
Two sprites out of sync	Relying on wait timing	Coordinate with <code>broadcast</code> / <code>when I receive</code>

A debugging helper in action: snap this under the green flag to see what the computer "thinks" the score is, then remove it when you're done.



◆ YOUR TURN – THE BUG HUNT (DEBUGGING BUDDIES)

Open `l3-3-bug-hunt-starter.sb3` (File → Load from your computer). It's the Catch the Apple game with **four bugs** hidden in it. You can't break it: the original can always be reloaded.

- 1 **Driver** holds the mouse. **Navigator** holds this workbook and asks: *What did you expect? What happened instead? Which block do you think is responsible? What's one thing we could change to test that?*
- 2 Play the game and **watch**. Try moving the mouse in different directions, watching whether the apple moves, missing on purpose, and pressing the green flag a second time.
- 3 For each bug, fill in one row below, then make **one** change and test. If it didn't help, undo it before trying the next idea.
- 4 **Swap roles** after every bug. Stuck? Ask the facilitator for a hint (there are three per bug).

Bug Hall of Fame

#	1 • WHAT WE WATCHED (SYMPTOM)	2 • BLOCK WE PREDICTED	3 • THE ONE CHANGE WE MADE	4 • FIXED?
1				☐
2				☐
3				☐
4				☐

◆ TEACH-IT PRACTICE – COACHING THROUGH FRUSTRATION

Rubber duck: have the child explain the code block by block, to you or a stuffed animal. **Ask, don't fix:** use the navigator's four questions. **Know when to break:** walking away is a real debugging strategy, not quitting.

SAY: *Walk me through it like I know nothing. What should this block do? ...And what did it actually do?*

SAY: *The code has a bug; you're a coder who's finding it.*

SAY: *Let's let our brains rest. Coders do this all the time.*

Role-play reflection: When did you most want to grab the mouse? What did you do instead?

◆ SELF - CHECK

Q1

The golden rule of debugging is...

- A) Change one thing at a time, then test
- B) Change many things at once to save time
- C) Delete everything and start over

Q2

What does "See Inside" do on a shared project?

- A) Opens the project's code to explore how it was made
- B) Deletes the project
- C) Charges money to view

Q3

Your child is melting down over a stubborn bug. Best move?

- A) Grab the mouse and fix it for them
- B) Have them explain it aloud, or take a short break
- C) Tell them to try harder

Q4

You click the green flag and *nothing* happens, though you're sure you built the script. Most likely suspect?

A) The computer is broken

B) The code is on a different sprite than you think

C) The backdrop is too dark

◆ LEARNING JOURNAL

Key concepts: Write the 4-step method from memory. Why does changing only one thing matter so much?

What clicked: Which "usual suspect" have you already met in your own projects? Did the table explain a bug you'd fixed by luck?

Progress & feelings: How do you react when code breaks, and has that changed since Module 1?

TRY AT HOME THIS WEEK

☐ Quietly move one block in your catch game and let your child debug it. Ask, don't fix. ☐ Open one shared project together with See Inside and borrow one idea. ☐ Start a "Bug Hall of Fame" page in your child's notebook. ☐ Keep the apple game, because its ideas return in the Module 4 capstone.

◆ NOTES

Leveling Up: Bigger Projects

In Session 4 you learn two power tools: clones (one sprite, many copies) and My Blocks (your own custom blocks). In Session 5, Family Build Night, you and your child use them to plan and build Star Catcher together, then show it off.

Lessons 4.1–4.4

4 lessons • 2 sessions

Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **Key Ideas**, a **Worked Example** drawn in Scratch blocks, **Your Turn** build steps, **Teach-It Practice**, a **Self-Check**, and a **Learning Journal**. The **Game Design Card** (page after Lesson 4.3's notes) is your homework between S4 and S5: fill it in *with your child*, and bring it to Family Build Night.

MODULE PROJECT TRACKER

- ◆ ☐ 4.1 (S4) — a sky of falling star clones, saved for the capstone
- ◆ ☐ 4.2 (S4) — custom blocks: **jump**, **dance (times)**, and **reset game**
- ◆ ☐ **Homework** — Game Design Card filled in with my child
- ◆ ☐ 4.3 (S5) — Star Catcher Part 1: bowl moves, stars fall, catching scores
- ◆ ☐ 4.4 (S5) — Star Catcher finished: lives, GAME OVER, launch day

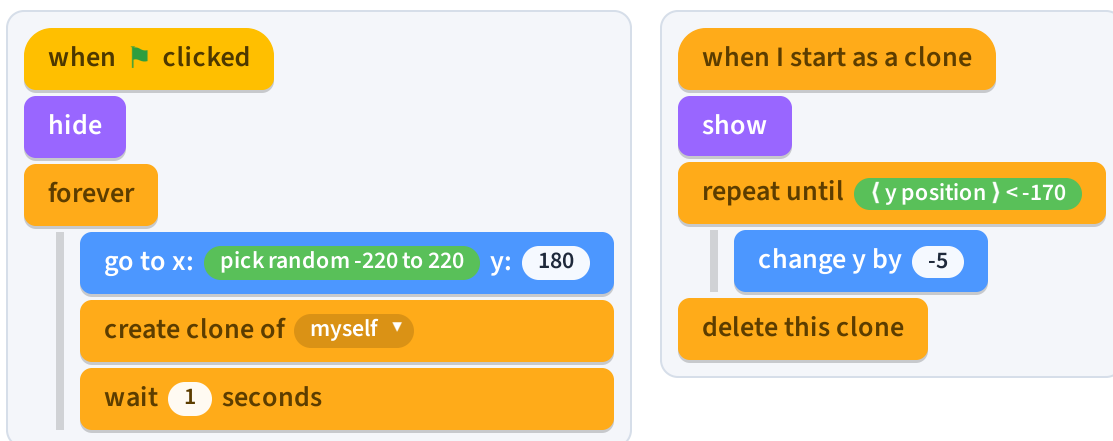
◆ BY THE END YOU CAN

- ◆ Explain a clone using the rubber-stamp idea
- ◆ Use **create clone of myself, when I start as a clone**, and **delete this clone**
- ◆ Build a hidden "stamp" that keeps making clones, and clean them up

◆ KEY IDEAS

- ◆ A **clone** is a copy of a sprite made *while the program runs*. The original is the **stamp**; clones are the **prints**.
- ◆ All three clone blocks are at the **very bottom of Control**.
- ◆ A clone copies everything, **including being hidden**. Start every clone script with **show**.
- ◆ **when clicked** runs only on the original. **when I start as a clone** runs only on each new clone.
- ◆ Scratch allows about **300 clones**. Every clone needs a way to reach **delete this clone**.

◆ WORKED EXAMPLE — THE STAMP AND THE PRINTS



Left: the stamp (the original, hidden). Right: what every print does. Both scripts go on the **same Star sprite**.

◆ YOUR TURN — FALLING STARS

- 1 New project. Hover the cat-face button (bottom right) → **Choose a Sprite** → **Star**. Pick a dark backdrop (Stars or Night City).
- 2 Build the stamp script. Click the green flag: **nothing appears, and that's correct**.
- 3 Build the clone script. The < block is in green Operators; drop blue **y position** (bottom of Motion) in its left slot and type -170.
- 4 Click the flag: stars rain down. Press stop: they all vanish. **Save it**.

- 5 Make it yours: **wait 0.3** for a meteor shower; **turn 5 degrees** inside the repeat; **set size to (pick random 30 to 100) %** before **show**.

◆ TEACH-IT PRACTICE – STAMPS AND PRINTS

Make it physical first (a rubber stamp, or a cookie cutter and play-dough). When the stamp script shows nothing, **don't fix it**. Ask:

SAY: *Where's the stamp now? In my hand, not on the paper. What should each print do after it lands?*

SAY: *We hid the stamp. The prints are copies of the stamp... so what do the prints look like right now?*

SAY: *Who should do the stamping — the stamp or the prints?*

After the role-play: which question helped your "child" find the bug? What did you want to do instead?

◆ SELF-CHECK

Q1

The original star is hidden. You make clones, but none appear on the stage. Why?

- A) Scratch only allows one clone at a time
- B) Clones start hidden, just like the original, and need a "show" block
- C) Clones must be switched on in the settings

Q2

Which script runs only on the new clones, never on the original?

- A) when  clicked
- B) when I receive [message]
- C) when I start as a clone

Q3

Your falling-stars game gets slower and slower, then new stars stop appearing. The most likely cause?

A) Clones are never deleted, so they pile up toward the 300 limit

B) The "wait 1 seconds" block is too long

C) The backdrop is too dark

Q4

A child puts create clone of myself inside the when I start as a clone script. What happens?

A) Exactly one extra clone appears

B) Every clone makes another clone, flooding the screen until Scratch's limit

C) Scratch shows an error message

◆ LEARNING JOURNAL

Key concepts: In your own words, how is a clone different from the original? What does each clone block do?

Questions & confusion: Which rule still feels slippery: what clones copy, which hats run where, or why deleting matters?

Ideas to try: What else comes in crowds (fish, balloons, falling pizza)? Sketch the stamp and the prints.

◆ NOTES

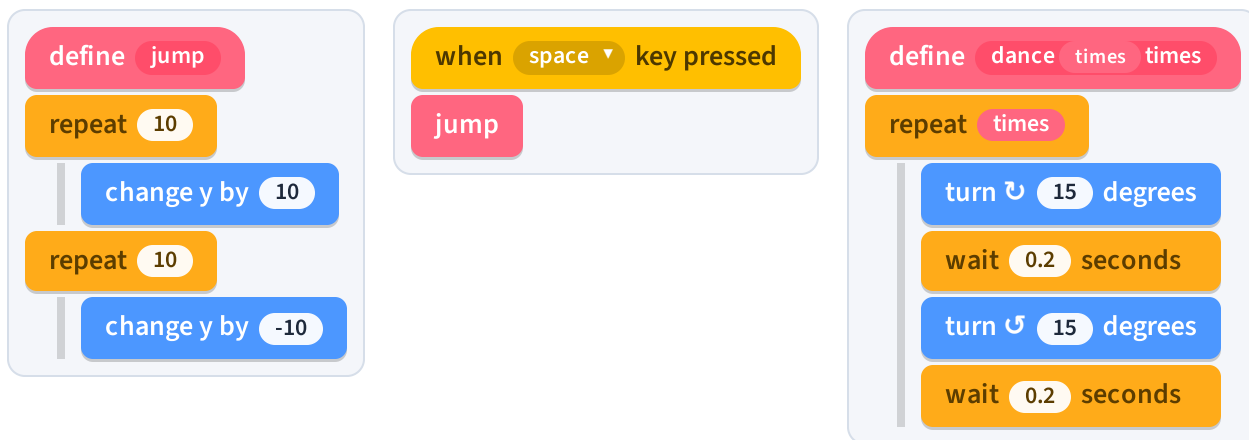
◆ BY THE END YOU CAN

- ◆ Make a custom block with **My Blocks → Make a Block**
- ◆ Put its steps under the pink **define** hat and use it from any script in that sprite
- ◆ Add an input so one block can behave differently each time

◆ KEY IDEAS

- ◆ A custom block is a **recipe card**: write the steps once, give them a name, then use the name. Programmers call it a **function**.
- ◆ Whatever is snapped under **define** is what the block does. Using the block is called **calling** it.
- ◆ An **input** is a slot you fill each time (e.g. how many times to dance). To use it, **drag the pink oval out of the define hat**; don't type the word.
- ◆ Leave **Run without screen refresh** unticked, or animations happen instantly.
- ◆ My Blocks belong to **one sprite**. Drag the define script onto another sprite to copy it.

◆ WORKED EXAMPLE — DEFINE ONCE, CALL ANYWHERE



◆ YOUR TURN — THREE CUSTOM BLOCKS

- 1 New project with the cat. **My Blocks → Make a Block → type jump → OK**. Build the define script above. Call it from **when space key pressed** *and* **when this sprite clicked**.
- 2 Make **dance**: **Add an input (number or text)** named **times**, then **Add a label "times."** Drag the **times** oval into the repeat. Call **dance 3 times**, **say Ta-da! for 1 seconds**, **dance 6 times** under the green flag.
- 3 Make variables **score** and **lives** (For all sprites). Then build **reset game**:



You'll use **reset game** again at Family Build Night. Challenge: **jump (height)**, using **(height / 10)** going up.

◆ TEACH-IT PRACTICE – NEW WORDS FOR THE COMPUTER

Start away from the screen with a "new word" game, then let your child find the repeated blocks and name the block themselves (silly names stick).

SAY: *I'm going to teach you a new word: zoomba. Zoomba means clap twice and spin around. Ready? Zoomba! Now let's teach the computer a new word.*

SAY: *I see those same blocks twice. What would you call what they do? What if that were one block with that name?*

SAY: *The value lives in this pink oval. Drag it out and see what happens.*

After the role-play: how did you respond to "copying is faster, why bother?"

◆ SELF-CHECK

Q1

Where do you put the steps that your new block should do?

A) Under the "when  clicked" block

B) Under the pink "define" block

C) Inside the Make a Block dialog

Q2

Your jump block works, but the cat seems to teleport instead of hopping. The likely cause?

A) "Run without screen refresh" is ticked

B) The block has no input

C) It's in the wrong sprite

Q3

You made a reset game block on the Bowl sprite, but it's missing when you click the Star sprite. Why?

A) The project wasn't saved

B) Block names can't have spaces

C) Custom blocks belong to the sprite they were made in

Q4

In define dance (times) times, how do you make the repeat use the number you pass in?

A) Type the word "times" into the repeat slot

B) Drag the pink times oval out of the define hat into the repeat slot

C) Make a new variable named times

◆ LEARNING JOURNAL

Key concepts: What's the difference between the define block and using the block? What does an input add?

Ideas to try: Name one block you wish Scratch had. Could you build it now?

Progress & feelings: How does it feel to make blocks instead of just using them?

◆ NOTES

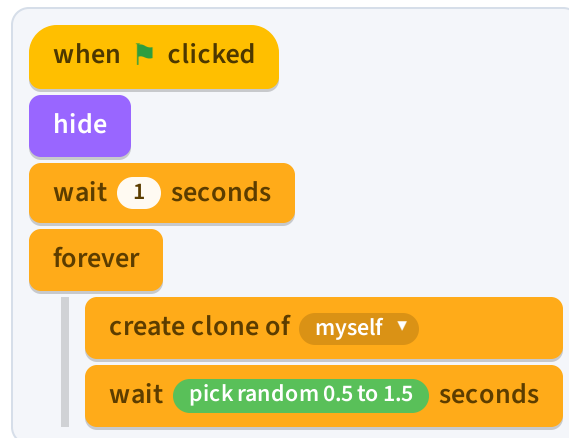
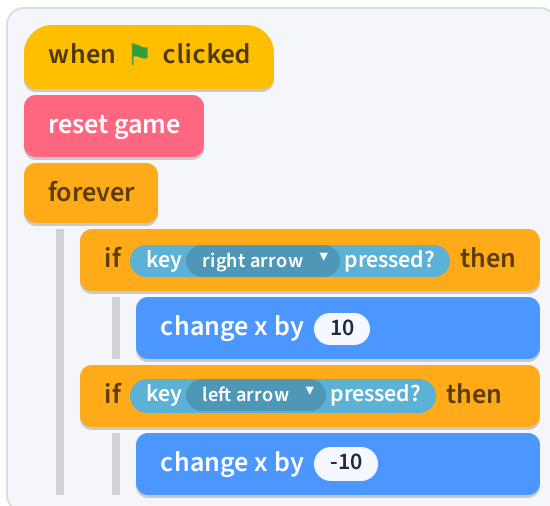
◆ BY THE END YOU CAN

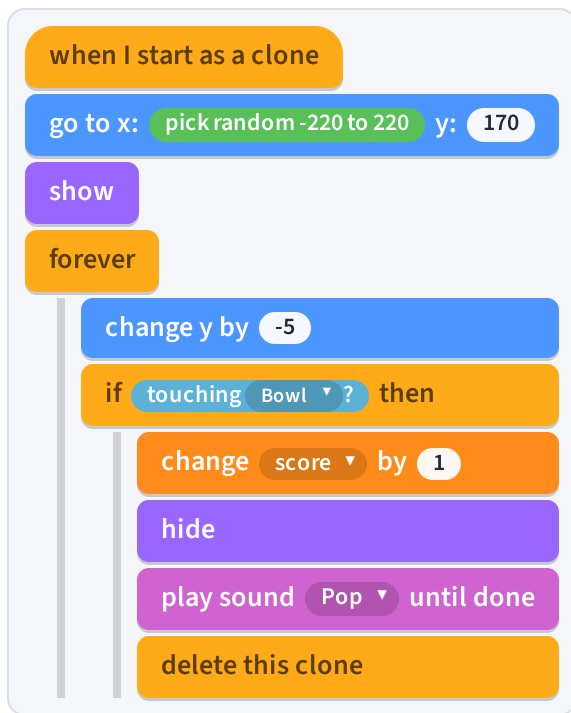
- ◆ Plan a game on paper with a Game Design Card and a Someday List
- ◆ Build the **smallest playable version** of Star Catcher first
- ◆ Share the work: your child **directs**, you **type** only when asked

◆ KEY IDEAS

- ◆ **Plan before you code:** brainstorm → pick ONE small idea → sketch it → list the pieces → build the smallest playable version → add one thing at a time.
- ◆ Big ideas that arrive mid-build go on the **Someday List**. "Not now" becomes "yes, later."
- ◆ The original Star is a **hidden factory**: it makes clones; only clones fall and get caught.
- ◆ **Test after every step.** If it worked a minute ago, the bug is in the last thing you added.
- ◆ A clone that is deleted stops its sound, so a caught star **hides, plays its sound until done**, then deletes itself.

◆ WORKED EXAMPLE — THE THREE CORE SCRIPTS





◆ YOUR TURN – STAR CATCHER PART 1 (PRESS THE FLAG AFTER EVERY STEP)

- 1 **Set the stage.** New project; right-click the cat → delete. Add **Bowl** and **Star**; pick the **Stars** backdrop.
- 2 **Variables.** Make **score** and **lives**, **For all sprites**, both ticked to show on stage.
- 3 **reset game on the Bowl.** Same block as Lesson 4.2 (set score 0, lives 3, go to x: 0 y: -150, show).
- 4 **Move the Bowl.** The left script above. Click the stage, then tap the arrows.
- 5 **Star factory.** The middle script, on the Star.
- 6 **Each clone falls.** The right script *without* the if. Stars pile up at the bottom for now; that's expected.
- 7 **Catch the stars.** Add the **Pop** sound (Sounds tab), then the **if touching Bowl?** part.

✓ CHECKPOINT – SMALLEST PLAYABLE VERSION

Stars fall, you catch them, the score climbs. That's a real game. Type **Star Catcher** in the title box and save (**File → Save now**, or **File → Save to your computer**). Behind? That's normal with a child driving; your facilitator has a rescue file so you can carry on together.

◆ TEACH-IT PRACTICE – BUILD IT AS A TEAM

Director (your child): decides what the game is, how it looks, and whether a change is right; presses every green flag. **Typist** (you): takes the mouse for fiddly bits, only when asked, narrates, and hands it straight back.

SAY: *You're the boss of this game. I'm your helper. Tell me what you want, and if you want me to do a tricky part, just say "your turn."*

SAY: Okay, I'm finding "change x by"... it's in blue Motion... snapping it inside the if. Your turn — press the flag and see if it worked!

SAY: Ooh, rocks too? Someday List! We'll add it once the stars work.

After Family Build Night: when was it hardest not to grab the mouse? What did you do instead?

◆ SELF - CHECK

Q1

Your child has a huge game idea with dragons, shops, and ten levels. What's the best first move?

- A) Open Scratch and start building all of it
- B) Pick the ONE main action, build that small version first, and save the rest for a "someday list"
- C) Tell them it's too hard and pick your own idea

Q2

In Star Catcher, why does the original Star sprite hide and only make clones?

- A) It's a hidden "factory." Only the clones fall and get caught
- B) Hidden sprites run faster
- C) So it can't make clones by accident

Q3

When a star is caught, why does the clone hide and play sound Pop until done before delete this clone?

- A) The score only counts after a sound plays
- B) Deleting a clone cuts off its sound, so it hides and finishes the pop first
- C) Scratch can't delete a visible clone

Q4

Which is the "smallest playable version" of Star Catcher?

A) Bowl moves + stars fall + catching adds score

B) The whole game, including lives and GAME OVER

C) The design card and sketch, before any code

◆ LEARNING JOURNAL

Key concepts: What is the "smallest playable version," and why build it first?

What clicked: Which piece fit better than expected: the star factory, **reset game**, or the catch check?

Progress & feelings: How did director/typist feel for you? For your child?

◆ NOTES



Fill this in **together**, in **your child's words**. Treat it as a conversation, not a form: "If you explained this game to your friend, what would you say?" Bring it to Family Build Night.

QUESTION

OUR ANSWER

Game name**Who is the player?****Controls:** how do you move?**What else moves?****How do you score?****How do you lose?****What happens at the end?****Smallest playable version****Someday list** (later ideas)

◆ SKETCH BOX — DRAW THE STAGE: WHERE THE PLAYER STARTS, WHAT MOVES, ARROWS FOR MOVEMENT

480 × 360 stage

◆ CHECKLIST OF PIECES (SPRITES, VARIABLES, RULES)

◆ ☐ _____

◆ BEST SCORES

◆ Designer: _____ score: _____

◆ Typist: _____ score: _____

- ◆ ☐ _____
- ◆ ☐ _____
- ◆ ☐ _____
- ◆ ☐ _____
- ◆ ☐ _____
- ◆ ☐ _____

◆ Launch-day player: _____ score: _____

Launch-day player liked:

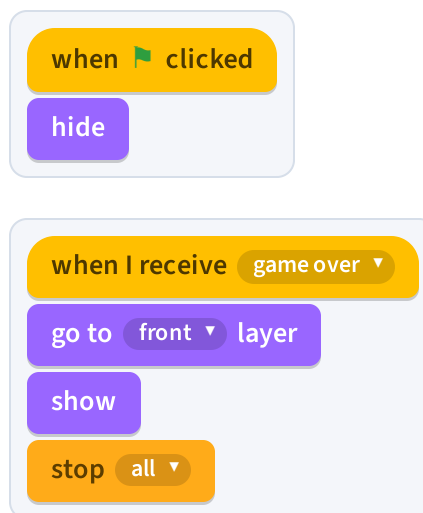
◆ BY THE END YOU CAN

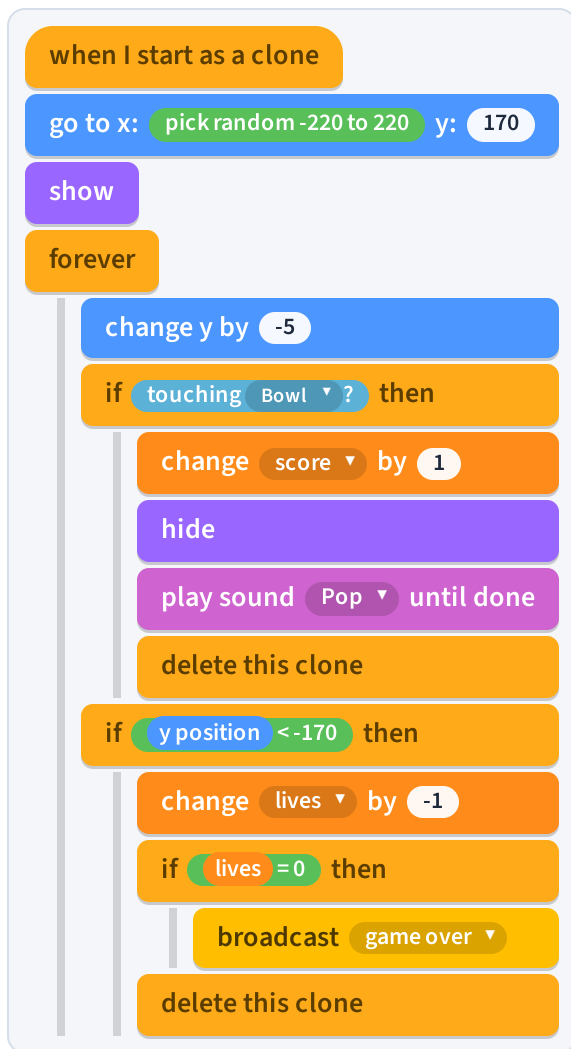
- ◆ Pick up a saved project by **playing it first**
- ◆ Use a lives check inside the clone and **broadcast game over**
- ◆ Show a GAME OVER sign, then **stop all**; tune the game until it feels right
- ◆ Hold a launch day where your child explains the game and watches someone play

◆ KEY IDEAS

- ◆ The **star** knows it was missed, so the star takes the life. **delete this clone** goes **last inside that if**, so each miss costs exactly one life.
- ◆ A **broadcast** lets one sprite tell another "it's over." The Game Over sprite **shows first**, then runs **stop all**, which freezes everything and removes all clones.
- ◆ **Tuning** numbers (fall speed, wait between stars) turns a working game into a fun one.
- ◆ Start by playing; **stop on a win**, never mid-bug.

◆ WORKED EXAMPLE — MISSES COST LIVES, THEN GAME OVER





◆ YOUR TURN – FINISH STAR CATCHER (STEPS 8–10)

- 1 **Play one round first.** Beat your score from before the break.
- 2 **Step 8 — a miss costs a life.** Add the second if inside the clone's forever, under the catch if. For **game over**, open the broadcast dropdown → **New message**.
- 3 **Step 9 — GAME OVER.** Paint a new sprite, write GAME OVER with the Text tool, name it **Game Over**, add its two scripts.
- 4 **Step 10 — music & tuning.** Stage → Sounds → a *Loops* sound; **when** ► **clicked** → **forever** → **play sound** (loop) **until done**. Too hard? -5 → -3. Too easy? The opposite.

✓ PROJECT COMPLETION

- ☐ Bowl moves left and right · ☐ Green flag resets score to 0 and lives to 3 · ☐ Original star hidden; clones fall from random spots · ☐ Catching adds 1 and plays a sound · ☐ A miss removes 1 life and the star disappears · ☐ At 0 lives GAME OVER appears and everything stops · ☐ Green flag starts a fresh game · ☐ Project saved

◆ TEACH-IT PRACTICE – FINISH STRONG & LAUNCH DAY

Launch Day: pick a player → the Director explains the rules first → **hands off**, don't explain for them → ask the player for one thing they liked and one idea (it goes on the Someday List).

SAY: *Before we add anything — can you beat your high score from last time? Show me!*

SAY: *It worked before we added this part, so the problem is only in this part. Let's read it together, one block at a time.*

SAY: *Let's save it and fix it with fresh eyes tomorrow. That's a real programmer move.*

Ask your child: "What was the hardest part, and how did you get past it?" Write their answer here, in their words:

◆ SELF-CHECK

Q1

Each missed star is taking away 2 or 3 lives instead of 1. What's the likely cause?

A) lives starts at 3 instead of 1

B) delete this clone isn't inside the lives if, so the star keeps subtracting

C) The Pop sound is too long

Q2

Why does the Game Over sprite show itself before running stop all?

A) So the sign is already on screen when everything freezes

B) It doesn't matter; the order is just style

C) Because stop all would otherwise make more clones

Q3

Right before the finish, the game breaks and your child is close to tears. Best move?

A) Quietly take the mouse and fix it for them

B) Point out it worked before the last change, look at just that part together, or save and stop for today

C) Start a new project from scratch

Q4

You press the green flag for a new game, but GAME OVER is still on the screen. The fix?

A) Move stop all above show

B) Set lives to 0 in reset game

C) Give the Game Over sprite a "when  clicked → hide" script

◆ LEARNING JOURNAL

Key concepts: What does the broadcast do in Star Catcher, and why does the lives check live inside the star clone?

Ideas to try: What did your launch-day player suggest? What would your child's next game be?

Progress & feelings: Looking back over Module 4, what are you proudest of? How did your child's confidence change from Part 1 to launch day?

◆ NOTES

YOU FINISHED MODULE 4 🎉

Clones, custom blocks, and a complete game taken from a paper plan to launch day with your child. **Try at home this week:** add one Someday List item together (one feature, test, save), or start a fresh Game Design Card for your child's own game. Next session: teaching Scratch to your child, starting with the art of not taking over.

Teaching Scratch to Your Child

You can code in Scratch now. This module is about the harder craft: helping your child learn without doing it for them, sharing safely, coaching a very young coder in ScratchJr, and choosing what comes next. It ends with a showcase and your graduation.

Lessons 5.1-5.4

4 lessons • Session S6

Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **Key Ideas**, a **Worked Example**, a **Your Turn** activity, **Teach-It Practice** (the say-this lines you'll rehearse in pairs), a **Self-Check**, a **Learning Journal** and **Notes**. There's very little coding here. The practice is in the role-plays and the journal, and nobody is grading your parenting.

MODULE PROJECT TRACKER

- ◆ ☐ 5.1 — coached a partner through a bug with my hands off the mouse
- ◆ ☐ 5.2 — completed the safety checklist and drafted our family sharing rules
- ◆ ☐ 5.3 — built "My Pet Says Hi" in ScratchJr (or planned it on paper)
- ◆ ☐ 5.4 — named our family's next project and where it leads on the road map
- ◆ ☐ S6 — final journal entry written; showcase and graduation 🎓

5.1 The Art of Not Taking Over

20 min

◆ BY THE END YOU CAN

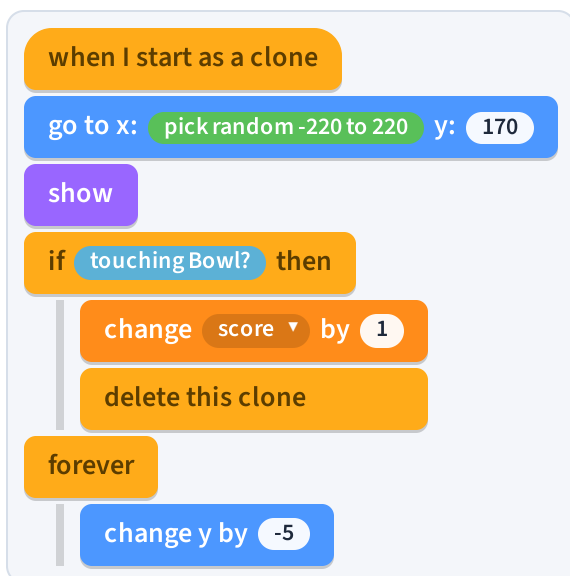
- ◆ Give the **least help that unblocks** your child, then fade it
- ◆ Ask a guiding question instead of giving the answer
- ◆ Praise effort and strategy, and teach the word "yet"

◆ KEY IDEAS

- ◆ **The struggle is the lesson.** Fixing it for them quietly says "this is too hard for you."
- ◆ **Scaffolding** is help that fades: *I lead → they lead, I prompt → they do it, I watch → they teach me or a sibling.*
- ◆ **Question toolkit:** What do you want to happen? What happened instead? Which block is in charge of that? What could we try to test it? Where have we seen this before?
- ◆ **Praise the process** ("You tested one block at a time"), not the talent ("You're a genius"). Add "...yet" to every "I can't."
- ◆ **Productive struggle:** bored → raise the bar; frustrated → shrink the step; boiling over → take a break (a real strategy, not a failure).

◆ WORKED EXAMPLE – COACHING A REAL BUG

Your child's Star Catcher stars fall, but the score never goes up. Here's the star clone's script:



You see it instantly: the *if touching Bowl?* check runs **once**, at the top of the screen, before the star falls. It belongs inside the *forever*. Don't say that. Climb the help ladder one rung at a time and stop as soon as your child is moving again:

- 1 "What do you want to happen when a star hits the bowl?"
- 2 "Which script is in charge of the score?"
- 3 "Let's read the star's script out loud. When does it check for the bowl?"
- 4 "What would happen if the check were inside the loop?"
- 5 *Last resort:* point to the *if* block and the *forever* block. They still do the dragging.

◆ YOUR TURN — HANDS-OFF COACHING ROUNDS

- 1 Pair up. The "child" draws a kid card and holds the mouse. The coach may not touch the mouse or trackpad.
- 2 Open the buggy project your facilitator provides. Coach for 4 minutes using only questions and process praise.
- 3 Swap roles, draw a new card, repeat.

◆ TEACH-IT PRACTICE

SAY: *What do you want to happen? ... And what happened instead?*

SAY: *How about I'm your hands and you're the boss? I'll only drag the blocks you point to.*

SAY: *You kept trying different things until it worked.*

SAY: *You can't make the stars fall... yet. Let's just get one to move first.*

Role-play reflection: the moment I most wanted to grab the mouse was... and what I said instead was...

◆ SELF-CHECK

Q1

What is "scaffolding" in teaching?

- A) Giving just enough help, then fading it as they gain confidence
- B) Doing the hard parts for them
- C) Leaving them entirely on their own

Q2

Which praise best builds resilience?

- A) "You're so smart!"
- B) "You kept trying different things until it worked."
- C) "See, that was easy!"

Q3

Your child is bored because the task is too easy. Best response?

- A) Offer a new challenge ("What if it kept score?")
- B) Make it simpler
- C) Stop the session

Q4

After a frustrating stretch, your child pushes the laptop at you: "Just do it for me!" Best response?

- A) Take the mouse and fix it quickly so they don't get upset
- B) Offer to be their "hands" while they're the "boss": you drag only the blocks they name
- C) Tell them to figure it out alone and leave the room

◆ LEARNING JOURNAL

Key concepts: In your own words, what is scaffolding? What's the difference between praising the process and praising the person?

What clicked: Think back to Family Build Night. Was there a moment you grabbed the mouse? What might a guiding question have sounded like instead?

Ideas to try: Two toolkit questions and one "yet" sentence for your sticky note on the computer:

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Set up a Scratch account the safe way (or decide not to need one yet)
- ◆ Explain "private until shared" and the community's protections
- ◆ Write family sharing rules and have the "before sharing" talk

◆ KEY IDEAS

- ◆ Scratch is run by a **non-profit**, collects minimal data, and asks for a **parent/guardian email**.
- ◆ Projects are **private until your child clicks Share**. Sharing is a choice, and an account is optional (File → Save to your computer works without one).
- ◆ The community is **moderated**, has short kid-readable **Community Guidelines**, and has **no private messaging**.
- ◆ Comment controls: **Report**, turn comments **off**, **delete** comments, report and block unwanted users.
- ◆ It's still the internet: shared device or common room, regular check-ins, and "show me any mean comment."

◆ WORKED EXAMPLE – ONE FAMILY'S SHARING RULES

THE RIVERA FAMILY SCRATCH RULES

1. Our username never has a real name, school, or birth year. (StarChef_77 ✓ · LeoMartinez2014 X)
2. No real name, address, school, phone number, or photos of us in projects or comments.
3. Scratch lives on the family laptop in the kitchen.
4. If anyone is ever unkind, we show a grown-up and handle it together: read it, report it, delete it.
5. When we remix, we say thank you in the Notes and Credits.

Signed: Leo _____ Mom _____

◆ YOUR TURN – SAFETY CHECKLIST & FAMILY RULES

- 1 Tick through the one-page **Scratch Safety Checklist** your facilitator hands out.
- 2 Draft three or four rules for your family below. At home, rewrite them with your child and both sign.

◆ TEACH-IT PRACTICE

SAY: *When we share your project, other kids around the world can see it and leave comments. Most are super nice! Our family rules: we never share private stuff, and if anyone's ever unkind, you show me and we handle it together. Deal?*

SAY: *Let's write a comment together: kind, curious, specific. "I love how the dragon breathes fire — how did you do that?"*

SAY: *Not everyone will love everything we make, and that's okay. We learn from helpful notes and ignore unkind ones.*

Role-play reflection: my scenario card was... and the line that worked best was...

◆ SELF-CHECK

Q1

When is a child's Scratch project visible to the public?

A) As soon as it's created

B) Only after they click "Share"

C) The moment it's saved

Q2

Which is TRUE about the Scratch community?

A) It's moderated, has guidelines, and has no private DMs

B) Kids can privately message anyone

C) Comments can't be controlled

Q3

Best safety habit for a young sharer?

A) Use their real full name as the username

B) A non-identifying username, your email on the account, and regular check-ins

C) Share everything publicly with no supervision

Q4

Your child gets an unkind comment on a shared project. What's the best first response?

A) Delete their account so it never happens again

B) Reply to the commenter and defend the project

C) Read it together, report it, delete it, and talk about how it felt

◆ LEARNING JOURNAL

What clicked: Which fact most eased (or sharpened) a worry you had: no private messaging, turning comments off, the Report button?

Progress & feelings: How do you feel about your child being part of an online community: excited, nervous, both? Which project would be a good first one to share together?

◆ NOTES

◆ BY THE END YOU CAN

- ◆ Explain how ScratchJr differs from Scratch
- ◆ Build a tap-to-talk pet with a recorded sound
- ◆ Run a short, playful session with a 4–7-year-old

◆ KEY IDEAS

- ◆ ScratchJr is a **free tablet app** (iPad, Android, Chromebook) with **picture blocks**, so no reading is needed.
- ◆ Blocks snap into a **row read left to right**, like a sentence. Motion numbers count **grid squares**.
- ◆ Six colors: **yellow** Triggering · **blue** Motion · **purple** Looks · **green** Sound · **orange** Control · **red** End. (Not the same as Scratch: trust the pictures.)
- ◆ Up to **four pages** per project, turned with **Go to Page**. Projects stay on the device: no online community.
- ◆ Coach in **10–15-minute** sessions: story first, you read, they tap, lots of silly sounds.

◆ WORKED EXAMPLE – "MY PET SAYS HI"



Start on Tap



Hop 2



Grow 2



My Sound



Shrink 2



End

Tap the pet: it hops, puffs up, says hi in a real voice, and settles back down. **Shrink** matters: without it the pet grows bigger with every tap. It's the same thinking as your first Scratch cat (an event, a sequence, a sound), laid out sideways. A rough Scratch equivalent stacks top to bottom (a "hop" becomes up, then back down):

when this sprite clicked

change y by 20

change size by 20

play sound recording1 until done

change size by -20

change y by -20

◆ YOUR TURN – BUILD IT ON A TABLET

- 1 Open ScratchJr → house icon → big +. Tap + beside the characters and pick an animal.
- 2 Yellow: drag **Start on Tap**. Blue: snap **Hop** on its right. Purple: **Grow**. Tap the pet to test.

- 3 Green: tap the microphone, record "Hi! I'm your pet!", accept, and snap the sound on.
- 4 Purple: **Shrink**. Red: **End**. Tap the pet.
- 5 *Stretch*: add page 2 (+ in the page strip) and end page 1's green-flag script with **Go to Page**.

No tablet? Plan a tiny two-page story your child could tell, one block per box:

◆ TEACH-IT PRACTICE

SAY: *Tell me a tiny story first. Who's in it, and what happens?*

SAY: *Which picture looks like jumping? ... Yes, the hop! What comes next?*

SAY: *Whoa, that's not what we planned! What should we try?*

SAY: *You made the puppy do that! You told it what to do, and it listened. That's what coders do.*

Role-play reflection: keeping my hands in my lap with a "five-year-old" felt... The thing that made them laugh was...

◆ SELF-CHECK

Q1

How are blocks arranged in a ScratchJr script?

A) Stacked top to bottom

B) In a horizontal row, read left to right

C) In any order; the app sorts them

Q2

You want the pet to react when your child touches it on the screen. Which block starts the script?

A) Start on Bump

B) Start on Green Flag

C) Start on Tap

Q3

What's the best length for a ScratchJr session with a 5-year-old?

A) About 10–15 minutes, ending while it's still fun

B) An hour, so they finish the whole project

C) Until they get frustrated, so they learn persistence

Q4

Every time your child taps the pet, it ends up bigger than before. What's the fix?

A) Add a Shrink block (same number) or Reset Size before End

B) Swap Start on Tap for Start on Green Flag

C) Add a second Grow block

◆ LEARNING JOURNAL

Key concepts: How is a ScratchJr row like a Scratch stack, and how is it different?

Questions & confusion: What tripped you up (the mismatched colors, pages, the microphone)? Check it before your child sits down.

◆ NOTES

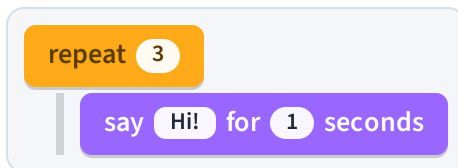
◆ BY THE END YOU CAN

- ◆ Place ScratchJr, Scratch, hardware and text code on one road map
- ◆ Name what's still ahead inside Scratch (lists, extensions)
- ◆ Explain why the jump to Python is mostly syntax

◆ KEY IDEAS

- ◆ **Road map:** ScratchJr (5–7) → Scratch (8+) → deeper Scratch, physical computing, or text languages. There's no single correct path.
- ◆ **Still ahead in Scratch:** **lists** (Variables → Make a List) and **extensions** via the **Add Extension** button, bottom-left: Pen, Music, Text to Speech, Video Sensing, micro:bit.
- ◆ **Beyond the screen:** Makey Makey (7+), LEGO robotics (8+), micro:bit (9+).
- ◆ **The concepts transfer.** Sequences, loops, conditionals, variables and events exist in every language.
- ◆ **Momentum comes from interest.** One more exciting project beats rushing to the next tool.

◆ WORKED EXAMPLE – THE SAME LOOP, TWO WAYS



```
for i in range(3):  
    print("Hi!")
```

Same loop, same count, same message. Your child already owns the idea; Python just types it.

◆ YOUR TURN – OUR NEXT PROJECT

What is your child excited to make next? Which branch of the road map does it lead to (deeper Scratch, hardware, text code)?

◆ TEACH-IT PRACTICE

SAY: Look how far you've come. Remember when we made the cat just say hello? Now look what you can build. What do you want to make next?

SAY: Could you add a second level?

SAY: *I don't know either. Let's find out together.*

Role-play reflection: how would you answer "Scratch is for babies, I want to do real coding"?

◆ SELF - CHECK

Q1

Your child built Star Catcher with clones and a My Block. What's a natural next thing to explore inside Scratch?

A) Nothing — they've learned all of Scratch

B) Lists, or an extension like Pen or Music

C) They must switch to Python right away

Q2

Why is moving from Scratch to Python not as scary as it sounds?

A) The core concepts transfer; only the syntax changes

B) Python and Scratch are exactly the same

C) You start completely from zero

Q3

Best way to keep a child's coding momentum?

A) Push them to the next tool as fast as possible

B) Follow their interests with another project they're excited about

C) Require an hour of coding daily

Q4

Where do you find the Pen and Music blocks in the Scratch editor?

A) In the Looks category

B) Behind the Add Extension button, bottom-left of the editor

C) They're only in the paid version

◆ LEARNING JOURNAL – YOUR FINAL ENTRY

First, flip back to your very first entry from Lesson 1.1 and read it slowly.

Your why: What did you hope to create with your child back then? Did you make it, or something even better?

Your worry: How big does it feel now? Cross it off if you've earned it.

Your number: In Lesson 1.1 you rated your confidence 1–10. What's your number today? What moved it most?

Progress & feelings: How has coding together changed your relationship with your child, or how they handle hard things?

◆ NOTES

WHAT NEXT — TAKE-HOME PAGE

TRUSTED NEXT RESOURCES (FROM LESSON 5.4)

Scratch Ideas & Tutorials — scratch.mit.edu/ideas · official, step-by-step, free

Scratch offline app — scratch.mit.edu/download · code without internet (Windows, Mac, ChromeOS, Android)

ScratchJr — for ages 5–7 (Lesson 5.3)

micro:bit — microbit.org · physical computing that works with Scratch

Code.org — free courses across ages, block-based and beyond

Scratch for Parents — scratch.mit.edu/parents · privacy and safety, straight from the source

Your local library or school — often runs free coding clubs. Ask!

IF YOUR CHILD LOVES...	TRY NEXT
Drawing & animation	The paint editor for original costumes; the Pen extension for a drawing toy
Music & noise	The Music extension (instruments, drums); a Makey Makey banana piano
Games	A second level, a boss, or a high-score list (Variables → Make a List)
Building things	micro:bit with the Scratch micro:bit extension; LEGO robotics
"Real" typed code	A first Python loop; tools that show blocks and text side by side
Teaching others	ScratchJr with a younger sibling or cousin; a school or library coding club

◆ OUR FAMILY'S NEXT PROJECT

KEEP BEING THE BUDDY

Even as they surpass you (they will!), your curiosity and your "show me what you made" stay just as valuable. You don't need to keep up with the code to keep cheering the coder. Every lesson stays online at rays-scratch.netlify.app to revisit whenever you like.